Informe final - PrestaMatch

Cicarelli Francisco

Eier Emanuel

Rossa Tomás



Universidad Católica de Santiago del Estero

Departamento Académico Rafaela

Ingeniería en Informática

Inteligencia Artificial

Rafaela

Octubre 2026

Informe final acerca del proceso de desarrollo de un modelo de machine learning para Optimización de Cartera P2P en "PrestaMatch":

Primero, repasamos la teoría presentada en las clases para que nuestro proceso de desarrollo fuese lo más fiel posible a lo enseñado durante la materia. Por lo tanto, comenzamos con un análisis exploratorio de los datos: obtuvimos información básica y estadística sobre los conjuntos de datos, verificamos que no hubieran datos duplicados, nulos o ilógicos y relacionamos algunas variables con la variable de interés para saber qué tanto podrían influir en la misma(con boxplots, crosstabs, etc) y probamos algunas hipótesis de esta forma. Al final de este proceso, consideramos que las variables que más podrían influir serían “máximo de días de atraso en alguna deuda”, “monto solicitado” y “tasa de interés”.

Luego, comenzamos con la etapa de train y validation usando las variables de estimación seleccionadas anteriormente. Separamos el dataset de train en train y validation para esto. De esta forma, no validaríamos con los mismos datos que se usaron para entrenar. Para esto, creamos un diccionario llamado “modelos” que almacena cuatro modelos de clasificación: Regresión Logística, K-Nearest Neighbors (KNN), Árbol de Decisión y Random Forest. Cada modelo se configura con sus respectivos parámetros. Mediante una función, entrenamos y evaluamos las predicciones de cada modelo en train y validation con métricas de interés que esta nos devuelve. Esto nos permitió estimar cuál de los modelos nos podría ser de más utilidad para el trabajo propuesto. No fueron tan precisas las estimaciones al final ya que Random Forest fue ampliamente superior y eso no se veía reflejado en las métricas. Sin embargo, no mostraba un mal rendimiento en las mismas. Al final de esta etapa, nos dimos cuenta que estábamos un poco limitados con las variables de estimación que estábamos usando así que, aparte de “máximo de días de atraso en alguna deuda”, usamos también “promedio de días de atraso en otras deudas”.

Una vez seleccionado el modelo final, se utiliza para estimar la probabilidad de que cada préstamo sea pagado. Estas probabilidades se utilizan para calcular la ganancia esperada de cada solicitud, teniendo en cuenta el monto solicitado, la tasa de interés y la propia probabilidad de pago. No pudimos reutilizar el mismo código para probar un modelo de redes neuronales así que la implementamos en un archivo aparte siguiendo, a excepción del filtro de ganancia positiva, la misma lógica(de todas formas, no tuvo un mejor rendimiento que la solución con Random Forest).

A continuación, se seleccionan únicamente los préstamos cuya ganancia esperada por dólar invertido es positiva. Estos candidatos se ordenan de mayor a menor según dicho indicador, priorizando aquellos que ofrecen una mayor rentabilidad esperada por cada dólar prestado.

Posteriormente, se establece un presupuesto máximo de 2.000.000 de dólares. Se recorren los préstamos ordenados y se aprueban aquellos que pueden financiarse sin superar el presupuesto disponible. Las solicitudes que no se seleccionan permanecen rechazadas.

Finalmente, se genera un archivo llamado submission.csv, que contiene el identificador de cada solicitud y la decisión de aprobación correspondiente. Este archivo se utiliza para entregar los resultados del modelo.

Obtuvimos varias soluciones cambiando diversos parámetros como n_estimators, max_depth y min_samples_leaf de Random Forest. También, en vez de calcular la ganancia esperada neta para ordenar los candidatos antes de elegir qué solicitudes aprobar, decidimos calcular la ganancia por cada dólar solicitado. Sin embargo, la lógica general cambió bastante poco a lo largo del desarrollo.